Uber :

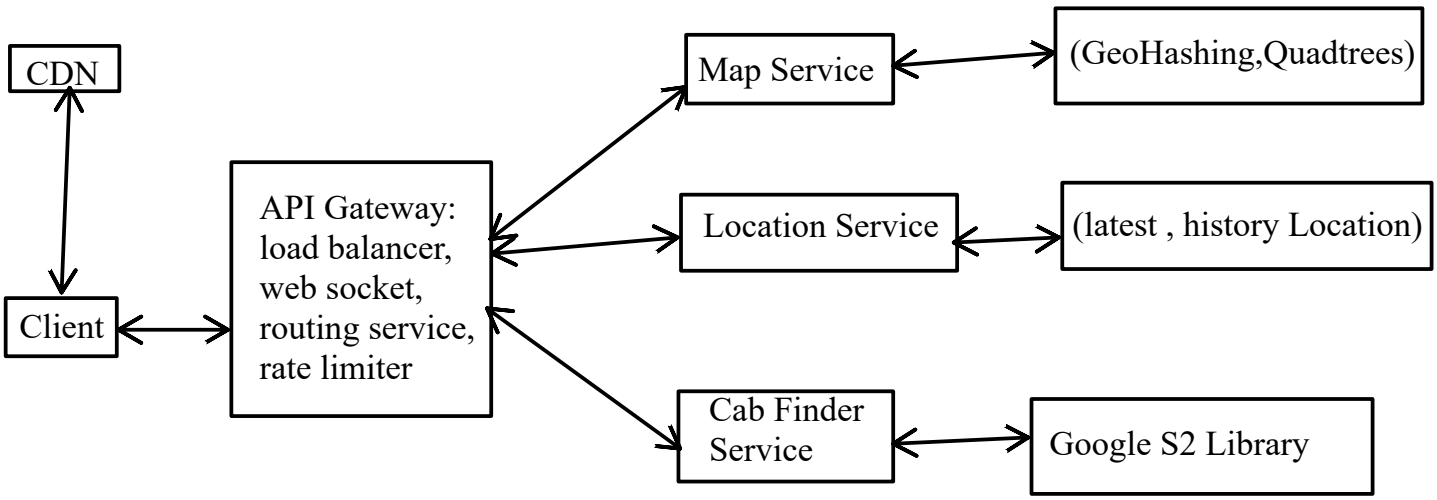
Functional requirements:
user should be able to book a cab based on his preference,
user should be able to search for any locations based on his preference,
rider should be able to accept/reject any ride based on his preference

Non-funtional requirements:
Data-partitioning,
load balancing.,
fault tolerance,
vertical scalability,
horizontal scalibility,
low latency,
high throughput,
CAP theorem(Here we need to prioritise Strong Consistency for booking Scenario in order to avoid double booking/double payment & for search scenario we need to prioritise 99.99% i.e Availability>>Consistency)

Core Entities:
user,
booking,
history,
activity,
payment

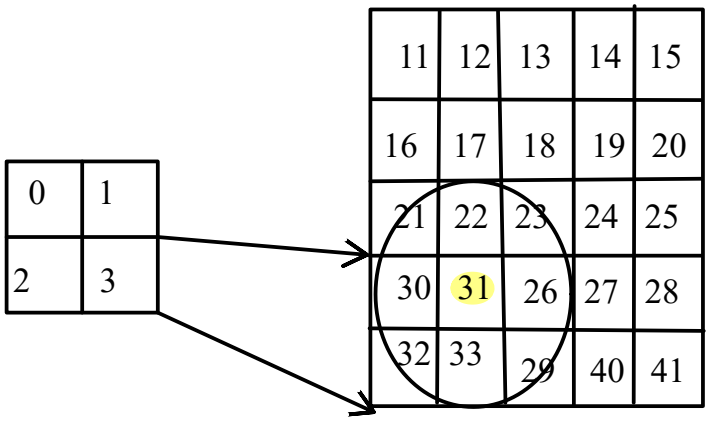
API :
POST:/cab/booking
{
    userID,
    {filemetadata}
}
GET:/search/{locationID}

HLD :

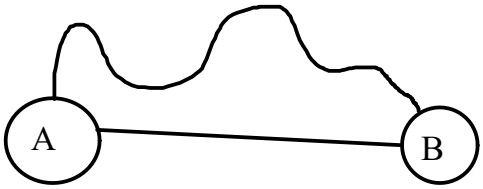


GeoHashing

(lat,long) -> chunkID



Google S2 Library



bcz we cannot use Euclidian distance to get the nodes which is present within vicinity of 5KM radius from the given chunkID ,

further through consistent hashing,gossip protocol all nodes will be getting the updated value